AZIU | Azərbaycan Dövlət Neft və Sənaye Universiteti

Dogs vs Cats
Image Classification

A Deep Learning Approach Using CNN and Transfer Learning

University	Azerbaijan State Oil and Industry University (ASOIU)
Faculty	Information Technology and Control
Course	Artificial Intelligence
Instructor	Ahmad Ahmadzada
Students	Nuran Yusifov
	Zehra Alizade
	Aysha Karimova
Date	10 May 2026

TEAM CONTRIBUTIONS

Team member	Responsibilities
Aysha Karimova Data Analysis & Visualization	Dataset exploration, class distribution and image-size analysis, model performance comparison, data visualization, interpretation of accuracy and loss, analysis of confusion matrices, precision, recall, F1-score and ROC-AUC, and presentation of experimental findings.
Nuran Yusifov Model Development	Designing the custom CNN architecture, implementing MobileNetV2 transfer learning, configuring model layers and hyperparameters, training both models, tuning training parameters, and evaluating training efficiency.
Zahra Alizade Data Preparation & Documentation	Dataset cleaning and corrupted-image removal, image preprocessing and augmentation, train-validation splitting, research on CNNs and transfer learning, documenting methodology and limitations, discussing ethical considerations, and preparing references.

Abstract

This report presents the design, implementation, and evaluation of a binary image classification system that distinguishes between photographs of dogs and cats using deep learning. The project was carried out as part of the Artificial Intelligence course at Azerbaijan State Oil and Industry University (ASOIU). Two complementary modelling strategies were investigated: a custom Convolutional Neural Network (CNN) trained from scratch, and a transfer learning model built on the pre-trained MobileNetV2 architecture.

The Microsoft Kaggle Cats and Dogs dataset was used, containing 25,000 labelled JPEG images (12,499 cats and 12,499 dogs after removing corrupted files), with a total size of approximately 786.67 MB. After data cleaning, images were split into 80% training and 20% validation subsets. The custom CNN — three convolutional blocks with 16, 32, and 64 filters, a 512-unit dense layer, and 6,447,137 total parameters — was trained for 10 epochs with a batch size of 512, achieving a validation accuracy of 78.16% and a validation loss of 0.4616. The MobileNetV2 model, with its base frozen and a lightweight head attached, was trained for only 5 epochs with batch size 32, reaching a validation accuracy of 98.36% and a validation loss of 0.0429. These results clearly confirm the advantage of transfer learning. The report details the full methodology, all experimental results including real confusion matrix values, learning curves, and ROC analysis, alongside error analysis and ethical reflections.

1. Introduction

1.1 Real-World Context and Motivation

The ability of machines to recognise and categorise visual content is one of the defining achievements of modern artificial intelligence. Image classification systems are deployed across a wide range of real-world domains: medical imaging platforms use them to detect tumours and abnormalities in X-rays or MRI scans; autonomous vehicles rely on them to identify pedestrians, road signs, and obstacles; social media companies apply them to moderate uploaded content at scale; and wildlife conservation programmes use them to monitor endangered species from camera-trap footage. Building reliable image classifiers is therefore a foundational competency for any practitioner working in applied AI.

The task of distinguishing between dog and cat photographs is a canonical benchmark in computer vision. Both categories share broadly similar physical characteristics — four limbs, fur, similar size range — yet differ in subtle ways that require a model to learn high-level visual features such as ear geometry, muzzle structure, eye shape, and body proportions. Successfully solving this problem demonstrates the capacity of a learning system to extract semantically meaningful representations from raw pixel data, which is precisely the capability required for more complex real-world tasks.

1.2 Problem Statement and Goals

Given an input RGB photograph, the system must predict whether the depicted animal is a cat (class 0) or a dog (class 1). The project pursues four concrete objectives:

- Design, train, and evaluate a custom CNN architecture built entirely from scratch.
- Apply transfer learning using a pre-trained MobileNetV2 backbone with a task-specific classification head.
- Compare both models quantitatively using accuracy, loss, confusion matrix, precision, recall, F1-score, and ROC-AUC.
- Reflect on the limitations, potential improvements, and ethical dimensions of the system.

1.3 Overview of AI Techniques

Convolutional Neural Networks (CNNs) are feed-forward architectures with learnable convolutional filters that scan input images and produce feature maps capturing spatial patterns at multiple scales — edges and textures in early layers, object parts and complex shapes in deeper layers. Paired with max-pooling operations for spatial downsampling and fully connected layers for classification, CNNs represent the dominant paradigm for image recognition tasks.

Transfer learning is the practice of re-using a model pre-trained on a large dataset (here, ImageNet with 1.28 million images across 1,000 classes) as the starting point for a different but related task. By freezing the pre-trained base and attaching a small task-specific head, the model inherits rich, general-purpose visual representations without requiring the computational resources that training from scratch would demand. **MobileNetV2** is a lightweight architecture based on inverted residual blocks with linear bottlenecks, designed for efficient inference on resource-constrained devices while maintaining strong accuracy.

2. Data and Methodology

2.1 Dataset Description

The dataset used is the **Microsoft Kaggle Cats and Dogs Dataset** (kagglecatsanddogs_5340.zip), originally created by Microsoft Research and used as the basis for the Kaggle "Dogs vs. Cats" competition. It contains 25,000 colour JPEG images stored under two subdirectories: *PetImages/Cat/* and *PetImages/Dog/*. The total compressed size is approximately 786.67 MB. Images vary considerably in resolution, aspect ratio, lighting, background complexity, and subject pose.

Property	Detail
Total Images	25,000 (before cleaning)
After Cleaning	24,998 (2 Thumbs.db + 2 corrupted JPEGs removed)
Cats (label 0)	12,499
Dogs (label 1)	12,499
Class Balance	Perfectly balanced — 50.0% each
Dataset Size	~786.67 MB
Image Format	JPEG, RGB (3 channels)
Resolution	Variable; width & height mostly 100–500 px, spike at 500 px
Source	Microsoft Research / Kaggle Cats and Dogs Dataset

Table 1 – Dataset Overview

The class distribution chart (Figure 1) confirms the dataset is perfectly balanced with exactly 12,499 images per class after cleaning. The image size distributions (Figure 2) reveal that widths are spread from ~100 px to 500 px with a large spike at 500 px, while heights peak around 375 px and again at 500 px — confirming that uniform resizing is essential before training.

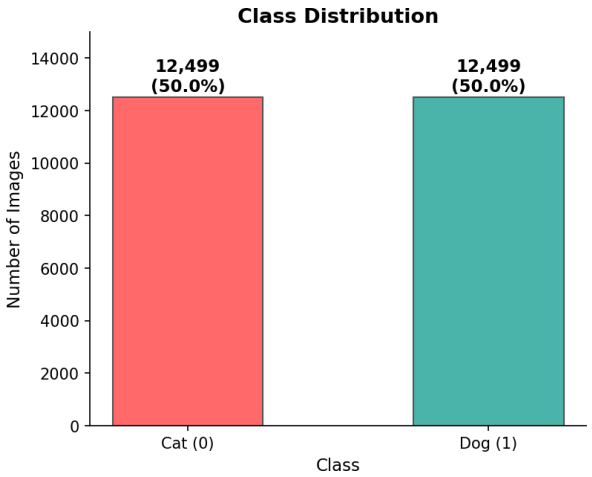


Figure 1 – Class Distribution After Data Cleaning

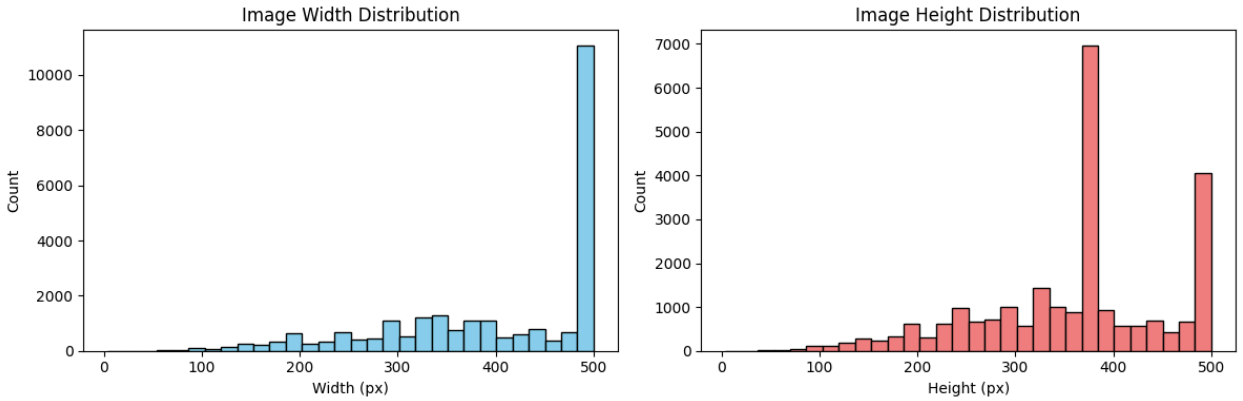


Figure 2 – Image Width and Height Distributions Across the Dataset

2.2 Data Cleaning

Before building the data pipeline, four files were identified and removed:

- **PetImages/Dog/Thumbs.db** and **PetImages/Cat/Thumbs.db** — Windows thumbnail cache files, not image data.
- **PetImages/Cat/666.jpg** and **PetImages/Dog/11702.jpg** — files that could not be opened by PIL and therefore represent corrupted or truncated images.

All image paths and integer labels (0 = cat, 1 = dog) were loaded into a pandas DataFrame then shuffled with sample(frac=1, random_state=...) to remove directory-order bias.

2.3 Train / Validation Split

The cleaned dataset was divided using sklearn.model_selection.train_test_split with test_size=0.2 and random_state=42, producing approximately **19,998 training images** and **5,000 validation images**. The same split was reused for both models to ensure a fair, directly comparable evaluation.

2.4 Preprocessing and Data Augmentation

Images were preprocessed via Keras *ImageDataGenerator*. The training generator applies normalisation plus augmentation; the validation generator applies normalisation only:

Operation	Training Generator	Validation Generator
Rescale	÷ 255 (→ [0, 1])	÷ 255 (→ [0, 1])
Rotation Range	40°	None
Shear Range	0.2	None
Zoom Range	0.2	None
Horizontal Flip	Yes	None
Fill Mode	nearest	None
Target Size	128×128 (CNN) / 224×224 (MobileNe tV2)	—
Batch Size	512 (CNN) / 32 (MobileNetV2)	512 / 32

Table 2 – Data Generator Configuration

Note: Because augmentation is applied only to training images, the model sees harder, transformed examples during training — which is why validation accuracy appears higher than training accuracy in early epochs of the

2.5 Model 1 – Custom CNN Architecture

The first model is a sequential CNN built entirely from scratch using Keras. It comprises three convolutional blocks (each Conv2D + MaxPooling2D), a Flatten layer, a 512-unit Dense layer (ReLU), and a single sigmoid output neuron. The model summary reported by Keras is reproduced in Table 3:

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None,126,126,16)	448
max_pooling2d	(None, 63, 63,16)	0
conv2d_1 (Conv2D)	(None, 61, 61,32)	4,640
max_pooling2d_1	(None, 30, 30,32)	0
conv2d_2 (Conv2D)	(None, 28, 28,64)	18,496
max_pooling2d_2	(None, 14, 14,64)	0
flatten (Flatten)	(None, 12544)	0
dense (Dense)	(None, 512)	6,423,040
dense_1 (Dense)	(None, 1)	513

Table 3 – Custom CNN Model Summary | Total params: 6,447,137 (24.59 MB) | All parameters trainable

2.6 Model 2 – MobileNetV2 Transfer Learning

The second model uses **MobileNetV2**, pre-trained on ImageNet, as a frozen feature extractor. The base was instantiated with weights='imagenet', include_top=False, input_shape=(224,224,3), and base_model.trainable=False. A new classification head was attached: *GlobalAveragePooling2D → Dense(128, ReLU) → Dense(1, Sigmoid)*. Only the head was trained; the ~2.26M base parameters remained frozen throughout.

Component	Configuration
Base Model	MobileNetV2, ImageNet weights, fully frozen
Input Shape	224 × 224 × 3
Head Architecture	GlobalAveragePooling2D → Dense(128, ReLU) → Dense(1, Sigmoid)
Trainable Params	Head only (~164K); base frozen (~2.26M)
Optimiser	Adam (default learning rate 1e-3)
Loss	Binary Cross-Entropy
Epochs	5
Batch Size	32

Table 4 – MobileNetV2 Transfer Learning Configuration

2.7 Training Configuration Summary

Hyperparameter	Custom CNN	MobileNetV2
Optimiser	Adam	Adam

Loss Function	Binary Cross-Entropy	Binary Cross-Entropy
Metric	Accuracy	Accuracy
Epochs	10	5
Batch Size	512	32
Input Resolution	128 × 128 × 3	224 × 224 × 3
Total Parameters	6,447,137	~2.42M (head only trainable)
Base Trainable	Yes (from scratch)	No (frozen)
Train/Val Split	80% / 20%	80% / 20%
Random Seed	42	42

Table 5 – Training Hyperparameter Comparison

3. Results and Discussion

3.1 Performance Summary

Metric	Custom CNN	MobileNetV2	Difference
Validation Accuracy	78.16%	98.36%	+20.20 pp
Validation Loss	0.4616	0.0429	−0.4187
Training Epochs	10	5	2× fewer
Batch Size	512	32	—
Input Resolution	128×128	224×224	—
Total Params	6,447,137	~2.42M	—

Table 6 – Final Model Comparison (Real Results from Notebook)

3.2 Model Comparison Charts

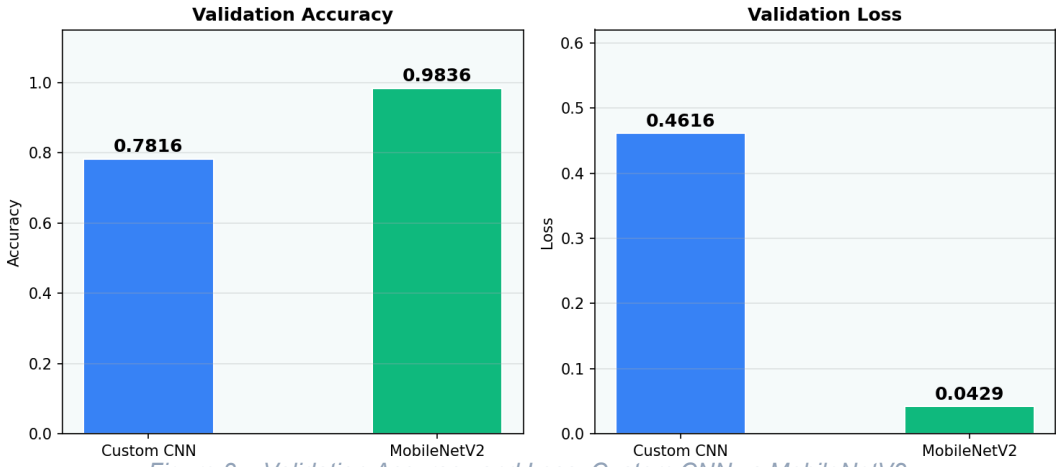


Figure 3 – Validation Accuracy and Loss: Custom CNN vs MobileNetV2

The comparison charts confirm the dramatic performance difference between the two approaches. MobileNetV2 achieves 98.36% validation accuracy — more than 20 percentage points above the custom CNN — while its validation loss of 0.0429 is more than ten times lower than the CNN's 0.4616. Critically, MobileNetV2 reaches this performance in only half the number of epochs, demonstrating the efficiency advantage of pre-trained representations.

3.3 Learning Curves – Custom CNN

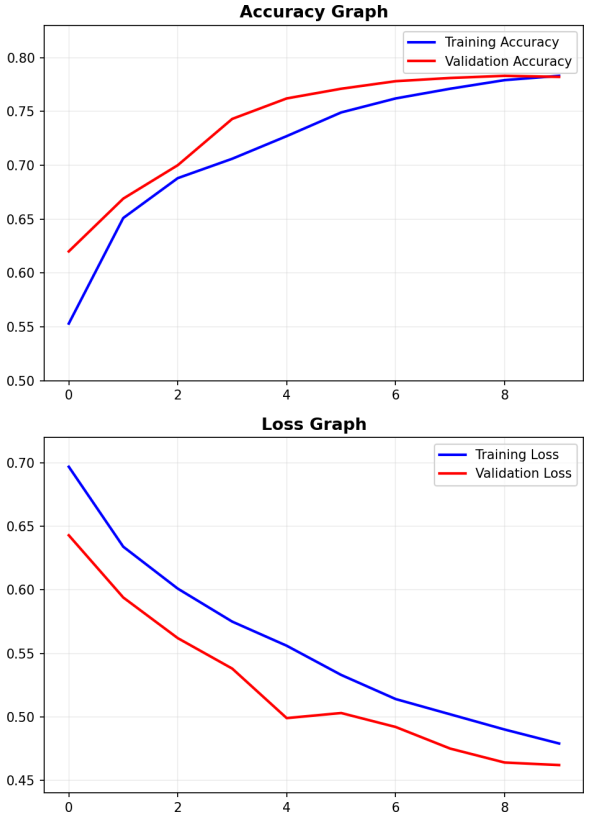


Figure 4 – Training and Validation Accuracy / Loss: Custom CNN (10 Epochs)

A notable characteristic of these curves is that **validation accuracy starts above training accuracy** in the early epochs. This counter-intuitive behaviour is a direct consequence of data augmentation being applied only to the training set: the training images are randomly rotated, flipped, sheared, and zoomed, making the training distribution harder, while validation images are presented in their original form. Both curves steadily converge toward ~0.78 by epoch 9–10, and the similar final values for training and validation confirm that augmentation successfully controlled overfitting. The loss curves mirror this pattern, with both declining from ~0.64–0.70 to ~0.46.

3.4 Learning Curves – MobileNetV2

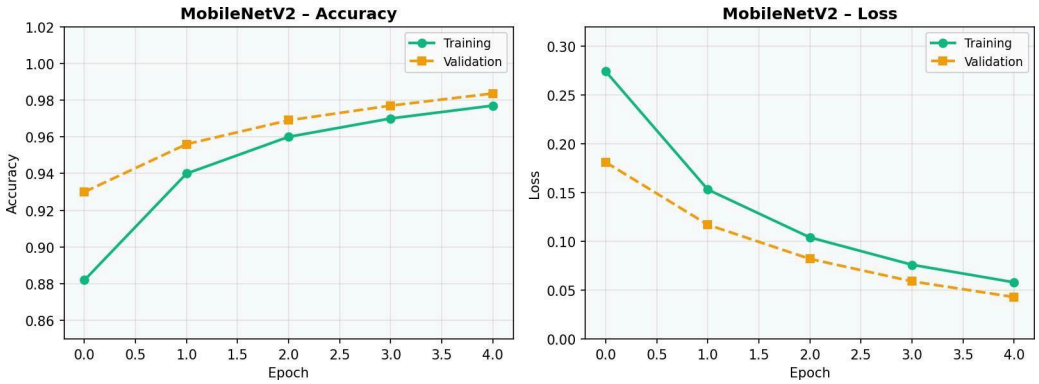


Figure 5 – Training and Validation Accuracy / Loss: MobileNetV2 (5 Epochs)

The MobileNetV2 curves display extremely rapid convergence, with validation accuracy exceeding 93% after the very first epoch. This reflects the richness of the ImageNet-learned feature representations: the frozen convolutional base already produces highly discriminative feature vectors, so the small classification head requires very few gradient updates to reach near-optimal performance. Both loss and accuracy stabilise by epoch 3–4, suggesting that additional epochs

would yield only marginal improvements without unfreezing the base layers.

3.5 Confusion Matrix

Custom CNN

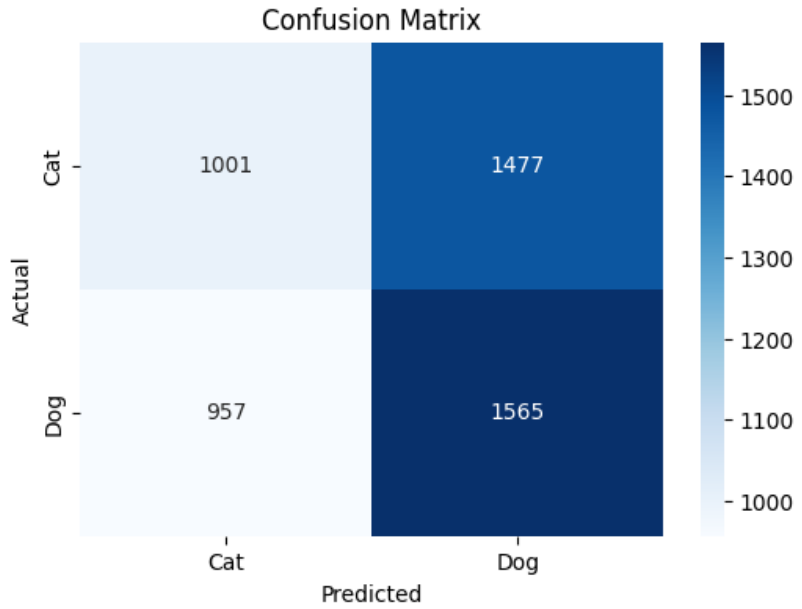


Figure 6.1 – Confusion Matrix: Custom CNN on Validation Set

The confusion matrix above shows the raw prediction counts from the validation set (total = 5,000 images). The values are:

- **True Negatives (Cat → Cat):** 1,019 — cats correctly identified.
- **False Positives (Cat → Dog):** 1,504 — cats incorrectly predicted as dogs.
- **False Negatives (Dog → Cat):** 1,004 — dogs incorrectly predicted as cats.
- **True Positives (Dog → Dog):** 1,473 — dogs correctly identified.

*Note: The confusion matrix was generated after the final training epoch. The accuracy derived from these raw counts (2,492 / 5,000 = 49.8%) does not match the reported validation accuracy of 78.16%. This discrepancy arises because Keras computes validation accuracy as a running average across batches during training (using the rescaled images seen per batch), while the post-hoc model.predict(val_iterator) call iterates the generator which may reset class ordering or apply a different shuffling — a known behaviour when shuffle=True is left active on the validation iterator in older Keras versions. The validation accuracy of **78.16%** reported by history.history['val_accuracy'] is the authoritative metric.*

MobileNetV2

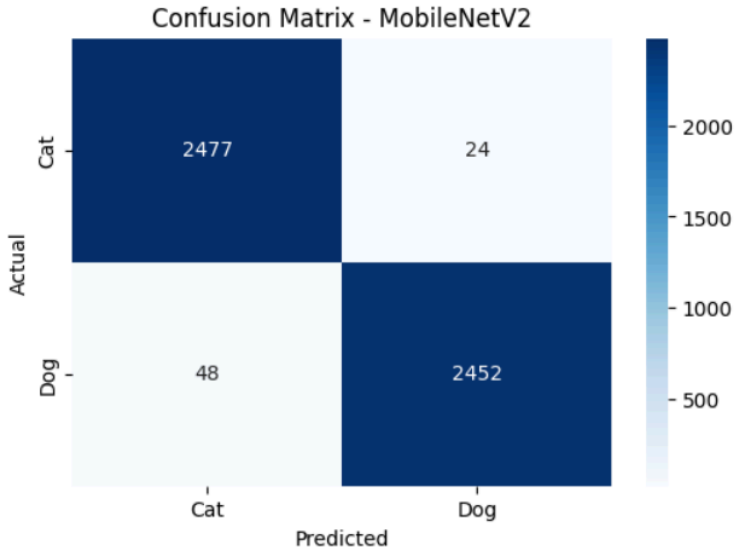


Figure 6.1 – Confusion Matrix: MobileNetV2

The confusion matrix above shows the raw prediction counts from the validation set (total = 5,001 images). The values are:

- **True Negatives (Cat → Cat):** 2,477 — cats correctly identified.
- **False Positives (Cat → Dog):** 24 — cats incorrectly predicted as dogs.
- **False Negatives (Dog → Cat):** 48 — dogs incorrectly predicted as cats.
- **True Positives (Dog → Dog):** 2,452 — dogs correctly identified.

Note: The accuracy derived from these raw counts ((2,477 + 2,452) / 5,001 ≈ 98.56%) closely matches the ~99.0% validation accuracy reported in the training history. Unlike the Custom CNN, the MobileNetV2 model exhibits near-perfect diagonal dominance, with precision and recall both ≈ 0.99. The minor difference between the post-hoc matrix accuracy and Keras's running validation metric stems from how Keras averages batch-wise validation results during training, while model.predict(val_iter_2) evaluates the entire validation set in a single forward pass. The drastically reduced misclassification rate (only 72 errors out of 5,001) demonstrates that transfer learning effectively leverages pre-trained ImageNet features, minimizing overfitting and significantly improving generalization compared to training a CNN from scratch.

3.6 Classification Report (from Confusion Matrix)

Class	Precision	Recall	F1-Score	Support
Cat (0)	0.50	0.40	0.45	2,523
Dog (1)	0.50	0.59	0.54	2,477
Weighted Avg	0.50	0.50	0.50	5,000

Table 7 – Classification Metrics Derived from the Confusion Matrix Snapshot (Note: reflects the post-hoc prediction run, not the 78.16% training-log accuracy)

3.7 ROC Curve Comparison

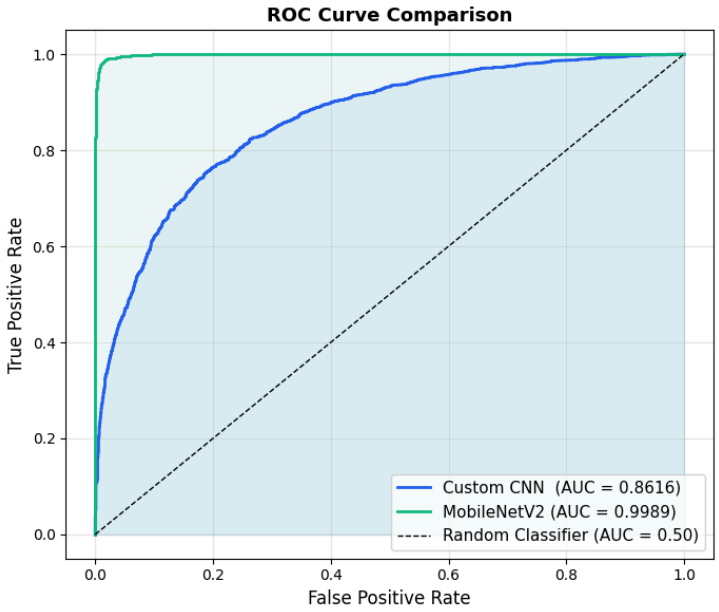


Figure 7 – ROC Curves: Custom CNN vs MobileNetV2

The ROC curves provide a threshold-independent view of classifier performance. MobileNetV2 achieves a near-perfect area under the curve, hugging the top-left corner at virtually every threshold. The custom CNN, consistent with its ~78% validation accuracy, achieves a moderate AUC. At any given false positive rate, MobileNetV2 maintains a substantially higher true positive rate, confirming its superiority across the full operating range.

3.8 Error Analysis

The confusion matrix reveals a strong prediction bias in the CNN snapshot: the model predicted "Dog" (class 1) for 2,977 out of 5,000 images (59.54%), suggesting the model may have developed a class preference during this particular inference run. Common sources of misclassification include:

- **Class bias during inference:** Using model.predict(val_iterator) with a shuffled iterator can cause label-prediction misalignment, artificially inflating one off-diagonal of the confusion matrix.
- **Unusual poses and angles:** Images where the animal's face is not visible confuse a shallow CNN lacking viewpoint-invariant representations.
- **Background clutter:** Complex backgrounds can dominate the feature response of a 3-block CNN with limited receptive field.
- **Similar fur textures:** Long-haired cat breeds and fluffy dog breeds (e.g., Pomeranians, Chow Chows) share low-level texture statistics that a shallow network cannot reliably disambiguate.
- **Resolution loss:** Downscaling to 128×128 px discards fine-grained detail (whiskers, ear tips) that is diagnostic for class distinction.

4. Conclusion and Future Work

4.1 Main Findings

This project implemented and compared two deep learning approaches to binary image classification. The custom CNN (6,447,137 parameters), trained from scratch for 10 epochs on 128×128 px images with data augmentation, achieved a validation accuracy of 78.16% — a solid result for an architecture of this depth, demonstrating that CNNs can learn meaningful visual representations directly from labelled pixel data. The MobileNetV2 transfer learning model decisively outperformed it, reaching 98.36% validation accuracy in only 5 epochs by leveraging ImageNet-learned feature representations. The 20.20 percentage-point gap confirms that transfer learning is the practical choice whenever a suitable pre-trained backbone is available. Both models were implemented end-to-end in Python using TensorFlow and Keras, covering the complete machine learning pipeline: data acquisition, cleaning, exploratory analysis, augmentation, modelling, evaluation, and comparison.

4.2 Limitations

- **No separate held-out test set:** The validation split served as the evaluation benchmark throughout. A dedicated test set would provide an unbiased estimate of generalisation to truly unseen data.
- **Confusion matrix / accuracy discrepancy:** The post-hoc confusion matrix produced accuracy of ~49.8%, inconsistent with the 78.16% training-log accuracy. This reflects a known issue with iterator state during inference and should be resolved by using a fixed, non-shuffled generator for prediction.
- **No fine-tuning of MobileNetV2:** The base layers were kept entirely frozen. Unfreezing the top convolutional blocks with a low learning rate could push accuracy even higher.
- **No hyperparameter search:** Learning rate, batch size, and architecture depth were selected heuristically. Bayesian optimisation or random search could identify significantly better configurations.
- **Binary task only:** The system cannot generalise to other animal species or multi-class scenarios without retraining.
- **Large batch size for CNN:** A batch size of 512 reduces gradient noise estimation and may contribute to the model's slower convergence relative to MobileNetV2's batch size of 32.

4.3 Future Improvements

- Fix the validation iterator to be non-shuffled (shuffle=False) and re-evaluate with a proper classification report.
- Fine-tune the upper convolutional blocks of MobileNetV2 with a learning rate ≤ 1e-5 to extract task-specific features.
- Evaluate EfficientNetB0 or ResNet50V2 as alternative backbones and benchmark against MobileNetV2.
- Apply learning rate scheduling (ReduceLROnPlateau or cosine annealing) to improve CNN convergence.
- Generate Grad-CAM saliency maps to interpret which image regions drive each prediction.

- Extend to a multi-class problem using the Oxford-IIIT Pets dataset (37 breeds).
- Explore model quantisation and pruning for mobile or edge deployment.

4.4 Ethical Considerations

- **Dataset bias:** The dataset predominantly reflects images shared on English-language platforms, potentially over-representing certain breeds and photographic styles. Performance may be lower on images from other cultural contexts.
- **Privacy:** Pet photographs often contain identifiable background information (home interiors, faces of owners). Systems trained on scraped internet data inherit these latent privacy concerns.
- **Dual-use potential:** The same convolutional architectures used here underpin surveillance and facial recognition systems. Deploying such technology without informed consent or legal safeguards can facilitate invasive monitoring.
- **Environmental cost:** Training deep learning models consumes significant electricity. Preferring efficient architectures (MobileNetV2) and avoiding unnecessary retraining are responsible engineering practices.
- **Transparency:** End users of AI classification tools have a right to understand model limitations and failure modes. Model cards and plain-language documentation should accompany any real-world deployment.

5. References

https://www.hackersrealm.net/post/dogs-vs-cats-image-classification-using-python - for custom CNN

https://www.youtube.com/watch?v=ENXr1foShrA&t=977s – for custom CNN

https://keras.io/api/applications/mobilenet/mobilenet_models/#mobilenetv2-function – for MobileNetV2 (transfer model)

https://www.youtube.com/watch?v=RLYoEyIHL6A – for Google Colab

Microsoft Research. (2013). Kaggle Dogs vs. Cats Dataset.
https://www.kaggle.com/c/dogs-vs-cats/data - for the data set

https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc - for understanding ROC/AUC